

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 25

Code of Conduct

I Athavan K(192524036)certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Athavan K

Assessment Tasks Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis
Corrective	25%	Proposes	Reasonable	Basic	Ineffective
Action		effective and feasible corrections	corrections	corrections	corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented

Question 1: *A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.*

1. Identification of Error

The root technical issue is input data skew combined with a poor InputSplit strategy. When splits are computed purely by HDFS block boundaries (default TextInputFormat) over a directory containing many small files or a few very large files, some map tasks receive disproportionately large chunks of data while others finish almost instantly.

Because MapReduce reducers begin their shuffle/copy phase only after all mappers finish producing output for a given partition, a single long-running "straggler" mapper forces every reducer to sit idle waiting for it. This is compounded if no combiner is used (each mapper emits full, unaggregated output, worsening shuffle volume) and if the number of reducers/partitions is mismatched to key distribution, so certain reduce tasks also receive far more keys than others (reducer-side skew).

2. Cause Analysis

- Small-files problem: many files smaller than the HDFS block size create excessive, unevenly-sized map tasks and per-task overhead.
- Skewed key distribution: a non-uniform partitioner (default hash partitioner on a skewed key) sends most records to one reducer.
- No use of CombineFileInputFormat, so splits map 1:1 to physical files/blocks instead of being logically balanced.
- Absence of a combiner increases the volume of intermediate data that the slow mapper must still process before emitting output.
- Speculative execution is disabled, so a genuinely slow/stuck mapper is never re-launched on a faster node.

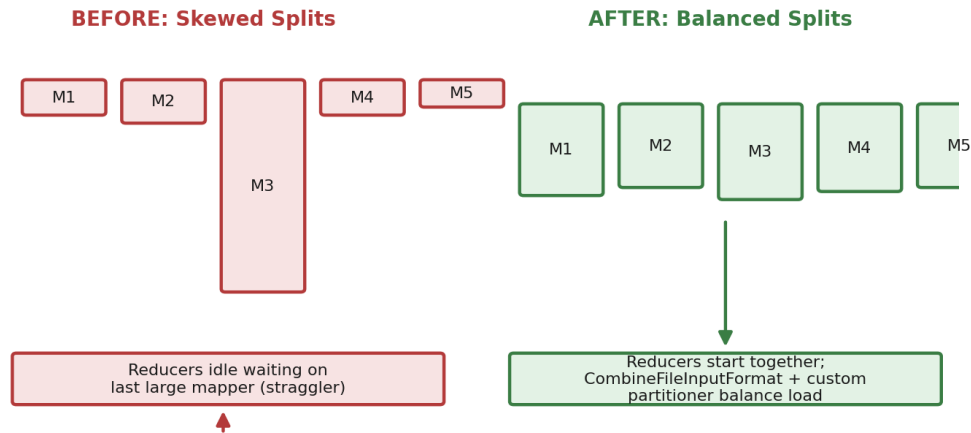
3. Corrective Action

- Use CombineFileInputFormat (or tune `mapreduce.input.fileinputformat.split.maxsize/minsize`) to merge small files and cap oversized splits into logically balanced units.
- Implement a custom Partitioner (or salt/hash skewed keys) so that reduce-side load is spread evenly instead of collapsing onto one reducer.
- Enable a Combiner to pre-aggregate mapper output locally, shrinking shuffle traffic and reducing the impact of any remaining skew.
- Turn on speculative execution (`mapreduce.map.speculative=true`) so straggler tasks are duplicated on idle nodes and the fastest result wins.
- Profile input with tools like Hadoop's `getmerge/HDFS` block reports beforehand and repartition/pre-aggregate heavily skewed source data at ingestion time.

4. Hadoop / Integration Concepts Applied

Applies MapReduce execution-model concepts correctly: InputSplit/RecordReader design, the mapper-then-shuffle-then-reduce dependency chain, Combiner optimisation, custom Partitioner design, and the YARN-level speculative execution mechanism — showing how split granularity directly drives reducer idle time.

Diagram



Question 2: *An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.*

1. Identification of Error

The design flaw is an under-replicated cluster (replication factor effectively set to 1 or 2 for critical data) combined with a NameNode that is not deployed in a High-Availability (HA) configuration. In this setup the NameNode is a Single Point of Failure (SPOF) for metadata, and any DataNode holding the sole copy of a block becomes a single point of failure for that block's data.

2. Cause Analysis

- `dfs.replication` is configured too low (e.g., 1) for the criticality of the data, so losing one DataNode makes those blocks completely unavailable.
- Rack awareness is not configured, so even at replication factor 3, all replicas may accidentally land on DataNodes in the same rack/switch, and a rack-level failure still causes unavailability.
- Only a single (Active) NameNode is running with no Standby NameNode, no JournalNodes/ZooKeeper Failover Controller — so a NameNode crash or restart halts the entire filesystem namespace, not just one node's data.
- No monitoring/alerting on under-replicated blocks means the cluster silently drifts into a fragile state before a failure exposes it.

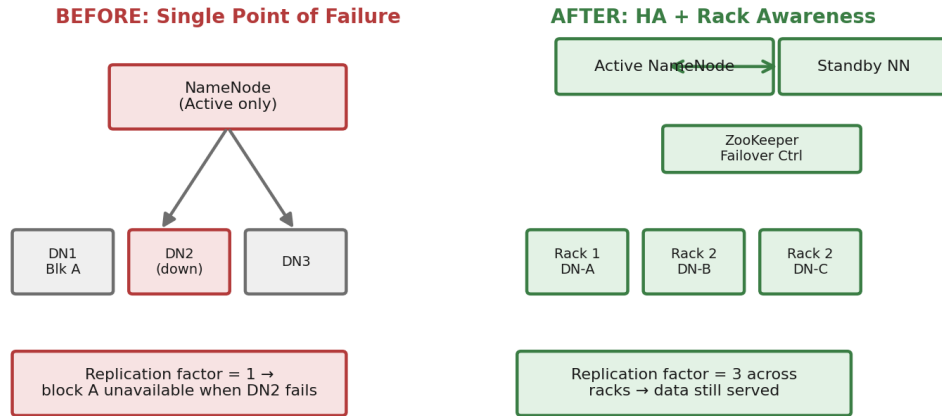
3. Corrective Action

- Set `dfs.replication` to at least 3 for production data, and use HDFS storage policies to guarantee critical datasets are never under-replicated.
- Enable rack awareness (`topology.script.file.name`) so the NameNode places replicas across at least two racks, tolerating a full rack failure.
- Deploy HDFS NameNode HA: an Active and Standby NameNode sharing edit logs through JournalNodes, coordinated by ZooKeeper's Failover Controller for automatic failover within seconds.
- Enable HDFS Federation for very large clusters to avoid a single namespace becoming a metadata bottleneck/SPOF.
- Add proactive monitoring for under-replicated/corrupt blocks (`hdfs fsck`, NameNode UI alerts) and automate re-replication.

4. Hadoop / Integration Concepts Applied

Demonstrates correct application of HDFS architecture: the NameNode/DataNode metadata-vs-data split, block replication placement policy, rack-awareness topology, and the HA NameNode + ZooKeeper failover mechanism — directly explaining why a single failure should never make data unavailable in a correctly designed cluster.

Diagram



Question 3: *A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.*

1. Identification of Error

The design error is a non-idempotent, at-least-once ingestion pipeline: NiFi processors retry on transient failures (timeouts, back-pressure, node restarts) and re-deliver flowfiles, while the downstream ETL/sink layer has no unique-key constraint or deduplication logic to safely absorb re-delivery. The result is the same logical record being written to the analytics store more than once.

2. Cause Analysis

- NiFi's default delivery guarantee is at-least-once; without idempotent writes at the sink, any retry (network blip, processor failure, provenance replay) reprocesses the same flowfile.
- No natural/business key or content hash is used to detect duplicates — inserts are plain append/INSERT operations rather than UPSERT/MERGE.
- Missing or misconfigured checkpointing means the pipeline cannot resume from the last successfully committed offset, so it restarts from an earlier point after a failure, reprocessing already-ingested records.
- Absence of a DetectDuplicate/dedup processor in the NiFi flow, and no deduplication window or watermark strategy in the ETL layer.
- Multiple concurrent NiFi flow instances or clustered nodes processing overlapping data without a distributed lock/state coordination.

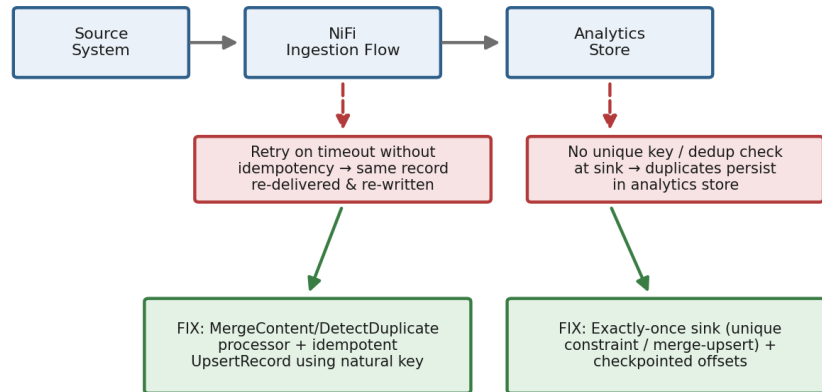
3. Corrective Action

- Introduce NiFi's DetectDuplicate processor (backed by a distributed cache) keyed on a record's natural key or content hash before data reaches the sink.
- Make the sink write idempotent — use UpsertRecord/MERGE statements keyed on a primary/business key instead of blind INSERTs.
- Enable checkpointing/offset tracking (e.g., cursor state in NiFi's state manager or the source connector) so restarts resume exactly where processing stopped.
- Apply exactly-once or effectively-once semantics at the storage layer (transactional writes, unique constraints) to make late/duplicate deliveries harmless.
- Add data-quality validation jobs that periodically scan for and reconcile duplicate keys as a safety net.

4. Hadoop / Integration Concepts Applied

Correctly applies big-data ingestion concepts: delivery-guarantee semantics (at-least-once vs exactly-once), idempotent sink design, NiFi flow-based processing with stateful processors, and checkpointing/offset management as used across Hadoop-ecosystem ingestion tools (Flume, NiFi, Sqoop, Kafka Connect).

Diagram



Question 4: *A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.*

1. Identification of Error

The scheduling error is the use of YARN's default FIFO Scheduler (or a Capacity/Fair Scheduler left in a single default queue with no per-user/per-team limits) in a multi-tenant cluster. A large, resource-hungry job submitted first acquires and holds the majority of containers, leaving concurrently submitted jobs from other users starved of CPU/memory until the first job completes.

2. Cause Analysis

- FIFO scheduling processes applications strictly in submission order with no notion of fairness or capacity guarantees across users.
- No queue hierarchy or capacity allocation exists to isolate teams/workloads, so one heavy job monopolises the ResourceManager's container allocation.
- No minimum/maximum resource limits (`yarn.scheduler.capacity.<queue>.capacity / maximum-capacity`) are configured per user or queue.
- Preemption is disabled, so once containers are granted to the large job, YARN cannot reclaim them for waiting jobs even if the queue exceeds its fair share.
- No user-level or application-level priority settings are applied to distinguish interactive/short jobs from long batch jobs.

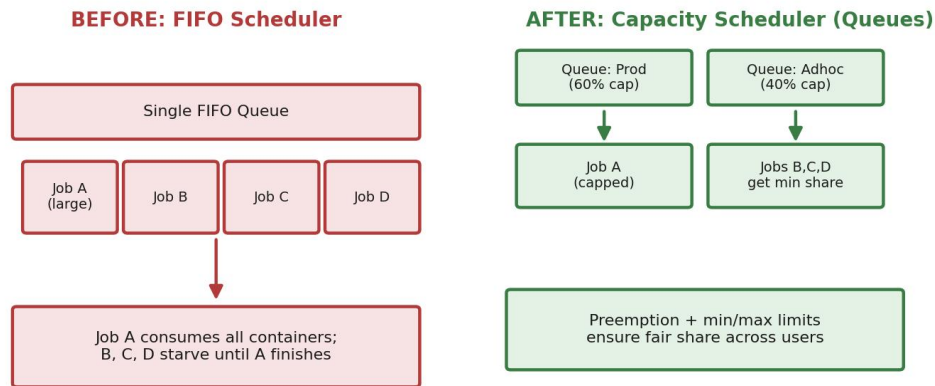
3. Corrective Action

- Replace FIFO with the Capacity Scheduler (or Fair Scheduler) and define separate queues per team/workload type with guaranteed minimum capacity and a maximum-capacity ceiling.
- Configure per-user limits within queues (`yarn.scheduler.capacity.<queue>.user-limit-factor`) to prevent any single user from consuming an entire queue's capacity.
- Enable preemption so containers from queues exceeding their guaranteed capacity can be reclaimed for queues that are under capacity.
- Set application priorities/labels to fast-track short interactive jobs ahead of long-running batch jobs where appropriate.
- Continuously monitor queue utilisation via the ResourceManager UI/metrics and rebalance capacity allocations as workload patterns change.

4. Hadoop / Integration Concepts Applied

Applies YARN's ResourceManager/Scheduler architecture correctly: the distinction between FIFO, Capacity, and Fair schedulers, hierarchical queue design, container preemption, and multi-tenant resource isolation — directly resolving the described starvation scenario.

Diagram



Question 5: *A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.*

1. Identification of Error

The design flaw is reliance on infrequent, asynchronous snapshot-based backups without adequate recency guarantees or restore validation. Because replication/backup runs on a fixed interval and lags behind live writes, any recovery performed after a failure restores the system to the last snapshot rather than the most recent consistent state — permanently losing all writes made between the snapshot and the failure.

2. Cause Analysis

- Backup/snapshot frequency (e.g., daily) is too coarse relative to the rate of data change, creating a wide Recovery Point Objective (RPO) gap.
- Replication to the backup/secondary store is asynchronous with no tracking of how far behind the replica is, so operators are unaware of the actual staleness at failure time.
- No write-ahead log or incremental change capture is used between snapshots, so changes since the last snapshot cannot be replayed during restore.
- No versioning/checksums on backups, so a corrupted or partially-written snapshot cannot be distinguished from a valid one.
- Restore procedures are not regularly tested end-to-end, so the staleness problem is discovered only during a real disaster, not in advance.

3. Corrective Action

- Increase snapshot/checkpoint frequency (or move to continuous/near-real-time replication) to shrink the RPO to an acceptable business threshold.
- Combine periodic full snapshots with incremental, log-based backups (write-ahead logs/journals) so changes after the last snapshot can be replayed during recovery.
- For critical data, use synchronous replication across at least two failure domains so no committed write can be lost even on immediate failure.
- Add checksums and version metadata to every backup artifact, and monitor replication lag with alerting thresholds.
- Conduct periodic recovery drills that measure actual RPO/RTO achieved, feeding results back into backup-frequency tuning.

4. Hadoop / Integration Concepts Applied

Applies distributed-storage reliability concepts correctly: RPO/RTO trade-offs, synchronous vs asynchronous replication, snapshot plus write-ahead-log recovery strategies (as used in HDFS snapshots, HBase WAL, and distributed backup tooling), and the need for restore validation in a Hadoop-ecosystem storage design.

Diagram

